AI110 | Foundations of AI Engineering

Foundations of AI Engineering Spring 2026 (@ Section 1F | Wednesdays 6PM - 8PM PT)
Personal Member ID#: 93101

Welcome
Getting Started
▶ Course Info
AI Prompting Guide
Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Week 7
Week 8
Week 9
Week 10
▶ Project Grading

Tinker: DocuBot

ℹ️ Activity Overview

⏰ **~45 minutes**

You've joined a development team that owns a growing software project with scattered, sometimes inconsistent documentation. The team dreams of having a small assistant that can answer questions like:

- "Where is the authentication token generated?"
- "Which endpoint returns all users?"
- "How do I connect to the database?"

Right now, a general purpose AI model has no idea what's inside your project's documentation folder, and it will happily guess if you let it. Your job is to build **DocuBot**, a lightweight retrieval-augmented assistant that can read your local docs, find relevant information, and generate grounded, reliable answers.

In this activity, you will explore three versions of the same system:

1. **Naive Generation**
   The LLM tries to answer questions using the entire documentation set at once, often producing fluent but unreliable responses.
2. **Retrieval Only**
   A simple search and ranking system you implement yourself, with no LLM involvement.
3. **Full RAG (Retrieval + Generation)**
   DocuBot retrieves the most relevant snippets first, then asks the LLM to answer using *only* those snippets, reducing hallucinations and improving accuracy.

By comparing these modes side-by-side, you'll see why retrieval matters and how it strengthens the reliability of AI systems operating on private or unfamiliar content.

🎯 Goals

By completing this activity, you will be able to…

- [ ] Explain the difference between naive generation, retrieval only, and RAG
- [ ] Implement a simple document index and retrieval pipeline
- [ ] Use retrieved snippets to ground LLM answers
- [ ] Add guardrails such as "I do not know" refusals when information is missing
- [ ] Evaluate DocuBot's behavior using realistic developer questions

🖍️ Tinker Instructions

▾ Part 1: Exploring the Knowledge Gap (Naive Generation)

⏰ **~10 mins**

In this phase, you'll configure LLM access, explore the DocuBot codebase, and observe how a naive AI system behaves when it lacks retrieval or guardrails. Your focus is not speed, but judgment.

- [ ] Go to the [DocuBot Starter Repo]() and fork it into your GitHub account.
- [ ] Clone your fork locally and open the folder in VS Code.
- [ ] Install dependencies:

```
pip install -r requirements.txt
```

- [ ] In the project root, open `.env.example`. This file shows the one environment variable DocuBot needs:

```
GEMINI_API_KEY=your_api_key_here
```

- [ ] Rename the `.env.example` file to `.env`.
- [ ] Visit the Gemini API key page (https://aistudio.google.com/app/api-keys) and create your own key. Copy the key into your `.env` file. The starter code will load this automatically using `python-dotenv`.
- [ ] To confirm your key is being read correctly, run:

```
python main.py
```

  If the key loads, you should see no warnings. If the key is missing, the program will let you know that LLM features are disabled.

- [ ] Open the `docs/` folder and skim the files. You don't need to understand every detail. Your goal is to build a rough mental model of what kinds of questions *can* be answered and what information is clearly *not* present.
- [ ] Open `docubot.py` and `main.py`. Use Copilot to select any code that feels unfamiliar or unclear. Decide whether you are trying to *understand*, *plan*, or *change* something, and use Copilot accordingly.
- [ ] Run DocuBot in **Naive LLM Mode** (Mode 1). Ask a few questions drawn from the docs folder:
  - "Where is the auth token generated?"
  - "Which endpoint returns all users?"
  - "How do I connect to the database?"
- [ ] Compare the answers to what you saw in the docs. Notice when the model sounds confident but vague, when it blends information from multiple files, and when it answers questions that the docs don't clearly support.

  > Treat these answers as hypotheses, not facts. At this stage, fluent output is a signal to investigate, not to trust.

📍 **Checkpoint:** You have configured LLM access, used Copilot to reason about an unfamiliar codebase, and observed the limits of naive generation. You are ready to design a retrieval system that gives the model evidence instead of guesswork.

Close Section

▾ Part 2: Designing and Implementing Retrieval

⏰ **~10 mins**

DocuBot can't answer questions reliably until it knows how to *search* your documentation. In this phase, you'll implement a simple retrieval system: an index that tracks where words appear, a scoring method that ranks relevance, and a function that returns the top matches. This is the core logic underlying both Retrieval Only mode and RAG mode.

- [ ] Open `docubot.py` and locate three methods that currently contain TODOs: `build_index`, `score_document`, `retrieve`.
- [ ] Read the docstrings carefully. Together, these functions define DocuBot's entire retrieval pipeline.
- [ ] Before writing any code, decide how you want this system to work. Use Copilot to help you reason about the design:
  - Are you trying to *understand* what the functions should do?
  - Are you trying to *plan* an approach?
  - Or are you ready to *edit* code?

  > Strong engineers separate *design* from *implementation*. Copilot is most useful here when it helps you think through tradeoffs before writing code.

- [ ] Implement simple versions of all three methods. A good first version of these uses basic Python, produces a numeric relevance score, and returns different results for different queries.

  > If Copilot suggest something you don't fully understand, pause. Ask it why it chose that approach, or simplify it until you can reason about every line.

- [ ] Run DocuBot in **Retrieval Only** mode:

```
python main.py
```

  Choose **Mode 2**.

- [ ] Try asking questions that appear in the docs, such as:
  - "Where is the auth token generated?"
  - "Which endpoint returns all users?"
- [ ] If results are confusing or incorrect, use Copilot on specific lines or functions to reason about the behavior and make targeted edits. Re-run the program after each meaningful change.

📍 **Checkpoint:** DocuBot now has a working retrieval system. You've used Copilot to plan, implement, critique, and refine core logic, while keeping yourself responsible for correctness and behavior.

Close Section

▾ Part 3: Improving Precision and Adding Safety

⏰ **~10 mins**

Your retrieval system works, but it is still rough. Right now, DocuBot often retrieves more text than necessary and may attempt to answer questions even when the documentation does not support an answer.

In this phase, you will:

- Refine retrieval to return smaller, more precise snippets
- Add guardrails so DocuBot knows when *not* to answer
- Use GitHub Copilot to refactor and reason about safety decisions

- [ ] Run DocuBot in **Retrieval Only mode (Mode 2)** and ask a few questions again. Notice when large chunks of irrelevant text are returned, important details are buried, the system tries to answer questions it shouldn't.
- [ ] Open `docubot.py` and identify where your retrieval logic decides *what text* gets returned. Before changing anything, pause and reason about the problem:
  - Is the issue happening before scoring, during scoring, or after sorting?
  - Is the unit of retrieval too large?

  Use Copilot to help analyze the existing code. Strong use of AI here means asking "why does this behave this way?" before asking "how do I change it?"

- [ ] Refactor your retrieval logic so DocuBot can identify *smaller sections* of text. You choose the strategy:
  - paragraphs
  - newline-separated sections
  - fixed-size slices
  - any simple, consistent rule

  Use Copilot to explore options and refine your approach. When collaborating with the AI:
  - Describe how you currently return whole documents
  - Explain how you want to extract smaller pieces
  - Ask it to suggest simple Python-only approaches
  - Select or adapt an approach that feels clear and maintainable
- [ ] Next, add a **guardrail** so DocuBot refuses to answer when it lacks meaningful evidence. Think carefully about what "no useful context" means in your system, where the decision to refuse should happen, and how to keep the logic explicit and readable. Use Copilot to reason about edge cases or refactor conditionals, but make the final call yourself.

  Refusal is a feature. A clear "I don't know" response is safer than a confident guess.

- [ ] Run DocuBot again in **Retrieval Only** mode and test a question clearly answered by the docs and a vague or unrelated question. Verify that relevant answers are more focused, and irrelevant questions trigger your guardrail.
- [ ] If behavior is surprising, inspect the logic manually and use Copilot on specific blocks to explain or simplify. Rerun after each meaningful change.

📍 **Checkpoint:** DocuBot now retrieves more precise snippets and refuses to answer when it lacks evidence. You've used Copilot to analyze, refactor, and improve reliability, while keeping yourself responsible for correctness and safety.

Close Section

▾ Part 4: Comparing Retrieval, Generation, and RAG

⏰ **~10 mins**

Now that DocuBot can retrieve precise snippets and safely refuse to answer, it's time for the final step: combining retrieval with generation. This is the heart of a RAG system. In this phase, you'll compare three versions of DocuBot side by side:

1. **Naive LLM Mode** — the model tries to answer using the entire docs corpus
2. **Retrieval Only** — no model, just your search logic
3. **RAG Mode** — retrieve first, then generate using only those snippets

Seeing these differences in action will help you understand why retrieval matters for grounding model outputs and reducing hallucinations.

- [ ] Open `main.py` and look at how the three modes are routed. You've already used Mode 1 and Mode 2 earlier. Mode 3 uses:
  - your retrieval logic
  - plus the LLM through the `GeminiClient`
  - plus strict rules that tell the model to use *only* the retrieved snippets

  > Improvements in RAG usually come from better system design, not a "smarter" model.

- [ ] Run DocuBot and test the same questions across all three modes. Keep the wording identical so your comparisons are meaningful. As you test, pay attention to what evidence each mode uses, what assumptions it makes, and when it should refuse to answer.
- [ ] Use Copilot selectively to help analyze differences. For example:
  - Explaining why two outputs differ
  - Identifying which snippets were actually used
  - Spotting unsupported claims
- [ ] Identify at least one example where:
  - Naive generation sounds confident but is weakly grounded
  - Retrieval only is accurate but hard to interpret
  - RAG balances clarity and evidence more effectively
  - Cases where RAG still fails
- [ ] Document your observations in `model_card.md`.

📍 **Checkpoint:** You have now experimented with all three versions of DocuBot and seen how retrieval and generation complement each other. You should have a clear sense of when an LLM needs retrieval to stay grounded and when retrieval alone is not enough.

Close Section